

# Django REST Framework (DRF) — Kirish Darsi Qo'llanmasi

API nima, nega kerak va birinchi ishlaydigan REST API yaratish

|               |                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------|
| Mavzu         | DRF bilan tanishuv — API tushunchasi, Serializer, ViewSet, Router                                 |
| Davomiyligi   | ~120 daqiqa (2 akademik soat)                                                                     |
| Oldingi bilim | Django asoslari: loyiha/app yaratish, view, urls.py, model va migratsiya                          |
| Dars maqsadi  | Har bir o'quvchi darsdan keyin ishlaydigan, JSON qaytaradigan to'liq CRUD API yarata olishi kerak |
| Natija        | Brauzerda ochiladigan Browsable API va 5 ta ishlaydigan endpoint                                  |

## 1. Dars rejasi (vaqt bo'yicha)

| Vaqt   | Mavzu                           | Faoliyat                                                                                           |
|--------|---------------------------------|----------------------------------------------------------------------------------------------------|
| 10 daq | Muammoni ko'rsatish             | Django HTML qaytaradi — lekin Android/React/bot uchun HTML foydasiz. Real misol bilan tushuntirish |
| 10 daq | API va JSON nima?               | API tushunchasi, JSON formati, REST prinsipi va HTTP metodlari (GET, POST, PUT, DELETE)            |
| 10 daq | DRF'siz qilib ko'rish           | Toza Django bilan JSON qaytarish — qancha kod va muammo chiqishini ko'rsatish                      |
| 5 daq  | DRF o'rnatish va sozlash        | pip install djangorestframework, INSTALLED_APPS ga qo'shish                                        |
| 10 daq | Model tayyorlash                | Product modeli yaratish va migratsiya qilish                                                       |
| 20 daq | Serializer — eng muhim qism     | ModelSerializer nima qiladi, ikki tomonlama tarjima va validatsiya                                 |
| 15 daq | APIView bilan birinchi endpoint | @api_view dekoratori, Response, request.data bilan ishlash                                         |
| 15 daq | ViewSet — 3 qatorda to'liq CRUD | ModelViewSet nima beradi, avvalgi kod bilan solishtirish                                           |
| 10 daq | Router va URL sozlash           | DefaultRouter avtomatik qanday URL yasaydi                                                         |
| 10 daq | Browsable API'ni sinash         | Brauzerda ochib, GET/POST'ni jonli test qilish                                                     |
| 15 daq | Amaliy mashq                    | O'quvchilar o'z modeli uchun mustaqil API yaratadi                                                 |
| 5 daq  | Yakunlash va uyga vazifa        | Asosiy tushunchalarni takrorlash, vazifa berish                                                    |

## 2. Nazariy asos — DRF nima uchun kerak?

### 2.1. Muammo: Django nima qaytaradi?

ASOSIY TUSHUNCHA

Oldingi darsda ko'rdik: Django HttpResponse orqali **HTML** qaytaradi. Ya'ni server tayyor sahifa yasab beradi, brauzer uni ko'rsatadi. Bu brauzer uchun mukammal ishlaydi.

Lekin zamonaviy loyihada ma'lumotga faqat brauzer emas, boshqa mijozlar (client) ham murojaat qiladi: **Android ilova, iOS ilova, React yoki Vue frontend, Telegram bot, boshqa serverlar**. Bularning birortasiga ham HTML kerak emas — Android ilovaga `<h1>Salom</h1>` yuborsak, u bilan hech narsa qila olmaydi.

Ularga kerak bo'lgan narsa — **toza, tuzilgan ma'lumot**.

```
Mijozga kerak bo'lgan javob — JSON

{
  "id": 1,
  "title": "Kapuchino",
  "price": "25000.00",
  "is_available": true
}
```

### 2.2. API, JSON va REST tushunchalari

#### NIMA UCHUN KERAK?

**API (Application Programming Interface)** — bu sayt emas, balki *ma'lumot xizmati*. U foydalanuvchiga ko'rinadigan chiroyli sahifa qaytarmaydi, balki dasturlar bir-biri bilan gaplashishi uchun ma'lumot qaytaradi.

**JSON** — ma'lumot almashish uchun standart matn formati. Deyarli barcha dasturlash tillari uni tushunadi, shuning uchun Python'da yozilgan server JSON yuborsa, Java'da yozilgan Android ilova ham, JavaScript'dagi React ham uni bimalol o'qiy oladi.

**REST** — API yaratishning keng qabul qilingan uslubi (qoidalar to'plami). Uning asosiy g'oyasi: har bir **manzil (URL)** biror **resursni** bildiradi, u bilan nima qilish kerakligini esa **HTTP metodi** aytadi.

| Metod  | Manzil (URL)     | Nima qiladi                                |
|--------|------------------|--------------------------------------------|
| GET    | /api/products/   | Barcha mahsulotlar ro'yxatini oladi        |
| GET    | /api/products/1/ | ID raqami 1 bo'lgan bitta mahsulotni oladi |
| POST   | /api/products/   | Yangi mahsulot qo'shadi                    |
| PUT    | /api/products/1/ | ID 1 bo'lgan mahsulotni to'liq yangilaydi  |
| PATCH  | /api/products/1/ | Uning faqat bir qismini yangilaydi         |
| DELETE | /api/products/1/ | ID 1 bo'lgan mahsulotni o'chiradi          |

**Doskada tushuntirish uchun:** manzil — *ot* (nima?), metod — *fe'* (nima qilamiz?). Shuning uchun REST manzillarida `/getProducts/` yoki `/deleteProduct/` kabi *fe'llar* yozilmaydi — *fe'l* vazifasini metodning o'zi bajaradi.

### 2.3. DRF'siz qilib ko'ramiz — muammoni his qilish

**NIMA UCHUN BU MUHIM?**

DRF'ni darrov o'rgatish o'rniga, avval uni **ishlatmasdan** qilib ko'rish kerak — shundagina o'quvchi DRF nimani yengillashtirayotganini haqiqatan his qiladi. Toza Django bilan JSON qaytarish mumkin, lekin har bir maydonni qo'lda yozib chiqish kerak.

#### FAQAT KO'RSATISH UCHUN — BU USUL BILAN DAVOM ETMAYMIZ

```
Toza Django bilan (tavsiya etilmaydi)

from django.http import JsonResponse

def product_list(request):
    products = Product.objects.all()
    data = []
    for p in products:
        data.append({
            'id': p.id,
            'title': p.title,
            'price': str(p.price),
            'is_available': p.is_available,
        })
    return JsonResponse(data, safe=False)
```

**BU YERDAGI MUAMMOLAR**

- Bu kod **faqat ro'yxatni ko'rsatadi** — qo'shish, tahrirlash, o'chirish uchun yana uch marta shunday yozish kerak.
- Kelgan ma'lumotni **tekshirish** (narx raqammi? nomi bo'sh emasmi?) — qo'lda yozilishi kerak.
- Xatolar uchun **to'g'ri status kod** (400, 404, 201) qo'lda belgilanadi.
- Model o'zgarsa (yangi maydon qo'shilsa), bu kodni ham qo'lda tuzatish kerak — unutilsa, xato chiqadi.
- Autentifikatsiya, ruxsatlar, sahifalash — hammasi noldan yoziladi.

**DRF aynan shu ishlarning barchasini tayyor holda beradi.**

## 3. Amaliy qism — birinchi API yaratish

### 1 DRF'ni o'rnatish

#### NIMA UCHUN KERAK?

DRF — Django'ning ichida tayyor kelmaydi, u alohida kutubxona. `pip install djangorestframework` uni internetdan yuklab, hozirgi faol virtual muhit (venv) ichiga o'rnatadi. Shundan keyin loyihada `rest_framework` nomi bilan import qilish mumkin bo'ladi.

#### AMALIYOT — TERMINALGA YOZING

```
pip install djangorestframework
```

### 2 settings.py ga qo'shish

#### NIMA UCHUN KERAK?

Django o'rnatilgan kutubxonani avtomatik ko'rmaydi — uni `INSTALLED_APPS` ro'yxatiga qo'shish kerak. Aynan shu qadam DRF'ning Browsable API sahifalari, shablonlari va statik fayllarini ishga tushiradi. Qo'shilmasa, brauzerda API ochilganda chiroyli interfeys o'rniga xato chiqadi.

#### CONFIG/SETTINGS.PY FAYLIDA

config/settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'rest_framework', # <-- DRF shu yerga qo'shiladi  
    'shop',           # <-- o'z appimiz  
]
```

### 3 Model yaratish va migratsiya

#### NIMA UCHUN KERAK?

**Model** — ma'lumotlar bazasidagi jadvalning Python ko'rinishi. API ma'lumot qaytarishi uchun, avvalo shu ma'lumot bazada saqlanadigan bo'lishi kerak. `makemigrations` — model o'zgarishlaridan "reja" (migratsiya fayli) tuzadi, `migrate` esa shu rejani bazaga qo'llab, haqiqiy jadvalni yaratadi.

#### SHOP/MODELS.PY FAYLIDA

shop/models.py

```
from django.db import models  
  
class Product(models.Model):  
    title = models.CharField(max_length=200)  
    price = models.DecimalField(max_digits=10, decimal_places=2)  
    is_available = models.BooleanField(default=True)  
    created_at = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.title
```

#### AMALIYOT — TERMINALGA YOZING

```
python manage.py makemigrations  
python manage.py migrate
```

### 4 Serializer — DRF'ning yuragi

#### NIMA UCHUN KERAK?

**Serializer** — ikki tomonlama tarjimon:

- **Chiqishda:** Python obyektini (model) JSON'ga o'giradi — mijozga yuborish uchun.
- **Kirishda:** kelgan JSON'ni Python obyektiga o'giradi va **avtomatik tekshiradi** (validatsiya): narx raqammi, nom juda uzun emasmi, majburiy maydonlar to'ldirilganmi.

Aynan shu **avtomatik validatsiya** DRF'ning eng katta foydasi — bu ishni qo'lda yozish juda ko'p vaqt oladi va xatoga moyil.

**ModelSerializer** — tayyor qisqartma: modelga qarab kerakli maydonlarni o'zi aniqlaydi. Model o'zgarsa, serializer ham avtomatik moslashadi.

## SHOP/SERIALIZERS.PY (YANGI FAYL YARATING)

```
shop/serializers.py

from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ['id', 'title', 'price', 'is_available', 'created_at']
```

**Solishtirib ko'rsating:** 2.3-bo'limdagi 12 qatorlik qo'lda yozilgan kod — endi shu 6 qatorga siqildi, ustiga validatsiya ham bepul qo'shildi. O'quvchilarga aynan shu taqqoslash eng kuchli ta'sir qiladi.

## 5 Birinchi endpoint — @api\_view bilan

### NIMA UCHUN KERAK?

**@api\_view** — oddiy Django view funksiyasini DRF view'iga aylantiruvchi dekorator. U ikkita muhim narsa beradi:

- `request.data` — kelgan JSON avtomatik Python lug'atiga (dict) o'girilgan holda.
- `Response()` — Python ma'lumotini avtomatik JSON'ga o'giradi va to'g'ri sarlavhalar bilan yuboradi.

Bu bosqichni ko'rsatish shart, chunki keyingi ViewSet'ning "sehr"i aslida nimani qisqartirayotganini o'quvchi shu yerda ko'radi.

## SHOP/VIEWS.PY FAYLIDA

```
shop/views.py

from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status

from .models import Product
from .serializers import ProductSerializer

@api_view(['GET', 'POST'])
def product_list(request):
    if request.method == 'GET':
        products = Product.objects.all()
        serializer = ProductSerializer(products, many=True)
        return Response(serializer.data)

    if request.method == 'POST':
        serializer = ProductSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

### KODNI QATOR-BAQATOR TUSHUNTIRISH

- `many=True` — bir nechta obyektini (ro'yxatni) o'girayotganimizni bildiradi. Bitta obyekt uchun yozilmaydi.
- `serializer.is_valid()` — kelgan ma'lumotni tekshiradi; xato bo'lsa `False` qaytaradi.
- `serializer.save()` — tekshiruvdan o'tgan ma'lumotni bazaga yozadi.
- `status=201` — "yangi narsa yaratildi" degan standart HTTP javobi; `400` — "yuborilgan ma'lumot noto'g'ri".

## 6 ViewSet — 3 qatorda to'liq CRUD

### NIMA UCHUN KERAK?

Yuqoridagi `@api_view` usuli ishlaydi, lekin faqat GET va POST'ni qamrab oldi. Bitta mahsulotni ko'rish, tahrirlash va o'chirish uchun yana shunga o'xshash ikkinchi funksiya yozish kerak bo'ladi — bu takrorlanuvchi kod.

**ModelViewSet** — shu takrorlanadigan barcha amallarni (ro'yxat, bitta element, qo'shish, to'liq yangilash, qisman yangilash, o'chirish) tayyor holda beradi. Bizga faqat ikkita narsani aytish qoladi: **qaysi ma'lumot** (queryset) va **qaysi serializer** orqali o'giriladi.

### SHOP/VIEWS.PY — YUQORIDAGI FUNKSIYA O'RNIGA

shop/views.py

```
from rest_framework.viewsets import ModelViewSet

from .models import Product
from .serializers import ProductSerializer

class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

**Doskada urg'u bering:** bu 3 qator kod avvalgi 20+ qatorlik funksiyaning hamma ishini bajaradi, ustiga yana PUT, PATCH va DELETE ham qo'shib beradi.

## 7 Router va URL sozlash

### NIMA UCHUN KERAK?

Oddiy Django'da har bir sahifa uchun `urls.py` ga qo'lda `path()` yozardik. `ViewSet`'da esa bitta klass bir nechta manzilga xizmat qiladi, shuning uchun ularni qo'lda yozish noqulay bo'lardi.

**Router** — `ViewSet`'ni ro'yxatdan o'tkazsak, unga kerakli barcha manzillarni **avtomatik** yasab beradi. Bu ham vaqtni tejaydi, ham manzillar REST standartiga mos bo'lishini kafolatlaydi.

### SHOP/URLS.PY (YANGI FAYL)

shop/urls.py

```
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

router = DefaultRouter()
router.register('products', ProductViewSet, basename='product')

urlpatterns = router.urls
```

### CONFIG/URLS.PY GA ULASH

config/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('shop.urls')),
]
```

### ROUTER AVTOMATIK YARATGAN MANZILLAR

GET `/api/products/` — ro'yxat | POST `/api/products/` — qo'shish  
GET `/api/products/1/` — bittasi | PUT va PATCH `/api/products/1/` — yangilash  
DELETE `/api/products/1/` — o'chirish

Bularning birortasi ham qo'lda yozilmadi — hammasini Router yasadi.

## 8 Serverni ishga tushirish va sinash

### BROWSABLE API — DRF'NING ENG TA'SIRLI TOMONI

DRF brauzerda API ochilganda chiroyli, interaktiv interfeys ko'rsatadi. U yerdan hech qanday qo'shimcha dastursiz (Postman kerak emas)

to'g'ridan-to'g'ri yangi mahsulot qo'shish, tahrirlash va o'chirishni sinab ko'rish mumkin. Darsda o'quvchilarga aynan shuni jonli ko'rsatish — eng katta taassurot qoldiradi.

AMALIYOT — TERMINALGA YOZING

```
python manage.py runserver
```

Brauzerda oching: **http://127.0.0.1:8000/api/products/**

DARSDA BIRGA SINAB KO'RING

- Sahifa pastidagi forma orqali yangi mahsulot qo'shing (POST) — ro'yxatda darrov paydo bo'ladi.
- Narx maydoniga ataylab matn (masalan sa'lom) yozib yuboring — DRF avtomatik xato qaytaradi. **Bu validatsiyaning jonli namoyishi.**
- /api/products/1/ manziliga o'ting va DELETE tugmasini bosib ko'ring.

## 4. Django va DRF — asosiy farq

| Jihat              | Django (oddiy)       | Django REST Framework                    |
|--------------------|----------------------|------------------------------------------|
| Nima qaytaradi     | HTML sahifa          | JSON ma'lumot                            |
| Kim ishlatadi      | Faqat brauzer        | Brauzer, mobil ilova, bot, boshqa server |
| Javob klassi       | HttpResponse         | Response                                 |
| Ma'lumot o'g'irish | Qo'lda (dict tuzish) | Serializer avtomatik                     |
| Tekshirish         | Form orqali          | Serializer avtomatik                     |
| Ko'rinish (view)   | Funksiya yoki CBV    | APIView, ViewSet                         |
| URL yozish         | Qo'lda path()        | Router avtomatik                         |

**Bir jumlada:** Django — HTML qaytaradigan *sayt* yasaydi. DRF — JSON qaytaradigan *API* yasaydi, uni kim ishlatishidan qat'i nazar. Zamonaviy loyihalarda backend (DRF) va frontend (React, mobil ilova) butunlay ajratilgan bo'ladi.

## 5. Ko'p uchraydigan xatolar va yechimlari

**ModuleNotFoundError: No module named 'rest\_framework'**

Sabab: DRF o'rnatilmagan yoki virtual muhit faollashtirilmagan. Yechim: `(env)` borligini tekshiring va `pip install djangorestframework` ni qayta bajaring.

**TemplateDoesNotExist: rest\_framework/api.html**

Sabab: `'rest_framework'` `INSTALLED_APPS` ro'yxatiga qo'shilmagan. Yechim: `settings.py` ni tekshiring va serverni qayta ishga tushiring.

**AssertionError: basename argument not specified**

Sabab: `ViewSet`'da `queryset` berilmagan yoki `Router`'da `basename` ko'rsatilmagan. Yechim: `router.register('products', ProductViewSet, basename='product')` shaklida yozing.

**Object of type Decimal is not JSON serializable**

Sabab: model maydonini qo'lda JSON'ga o'g'irishga urinilgan. Yechim: `Serializer` ishlatib — u `DecimalField`, `DateTimeField` kabi maydonlarni o'zi to'g'ri o'giradi.

**O'qituvchi uchun maslahat:** darsni albatta 2.3-bo'limdan (DRF'siz qo'lda yozish) boshlang. O'quvchi avval muammoni his qilmasa, `Serializer` va `ViewSet` unga shunchaki "yodlash kerak bo'lgan sintaksis" bo'lib tuyuladi. Muammoni ko'rgandan keyin esa yechim mantiqiy va esda qoladigan bo'ladi.

## 6. Uyga vazifa

- Loyihaga **Category** nomli yangi model qo'shing (maydonlar: `name`, `created_at`) va migratsiya qiling.
- `Product` modeliga `category` nomli `ForeignKey` maydon qo'shing.
- `CategorySerializer` va `CategoryViewSet` yozib, `Router`'ga ro'yxatdan o'tkazing.
- `Browsable API` orqali kamida 3 ta kategoriya va 5 ta mahsulot qo'shing.
- Qo'shimcha (ixtiyoriy):** `ProductSerializer` ichida `category` maydoni faqat ID emas, balki kategoriya nomini ham ko'rsatadigan qilib o'zgartiring (maslahat: `StringRelatedField` yoki ichma-ich `serializer`).
- Barcha endpointlarni sinab, natijalarni skrinshot qilib yuboring.